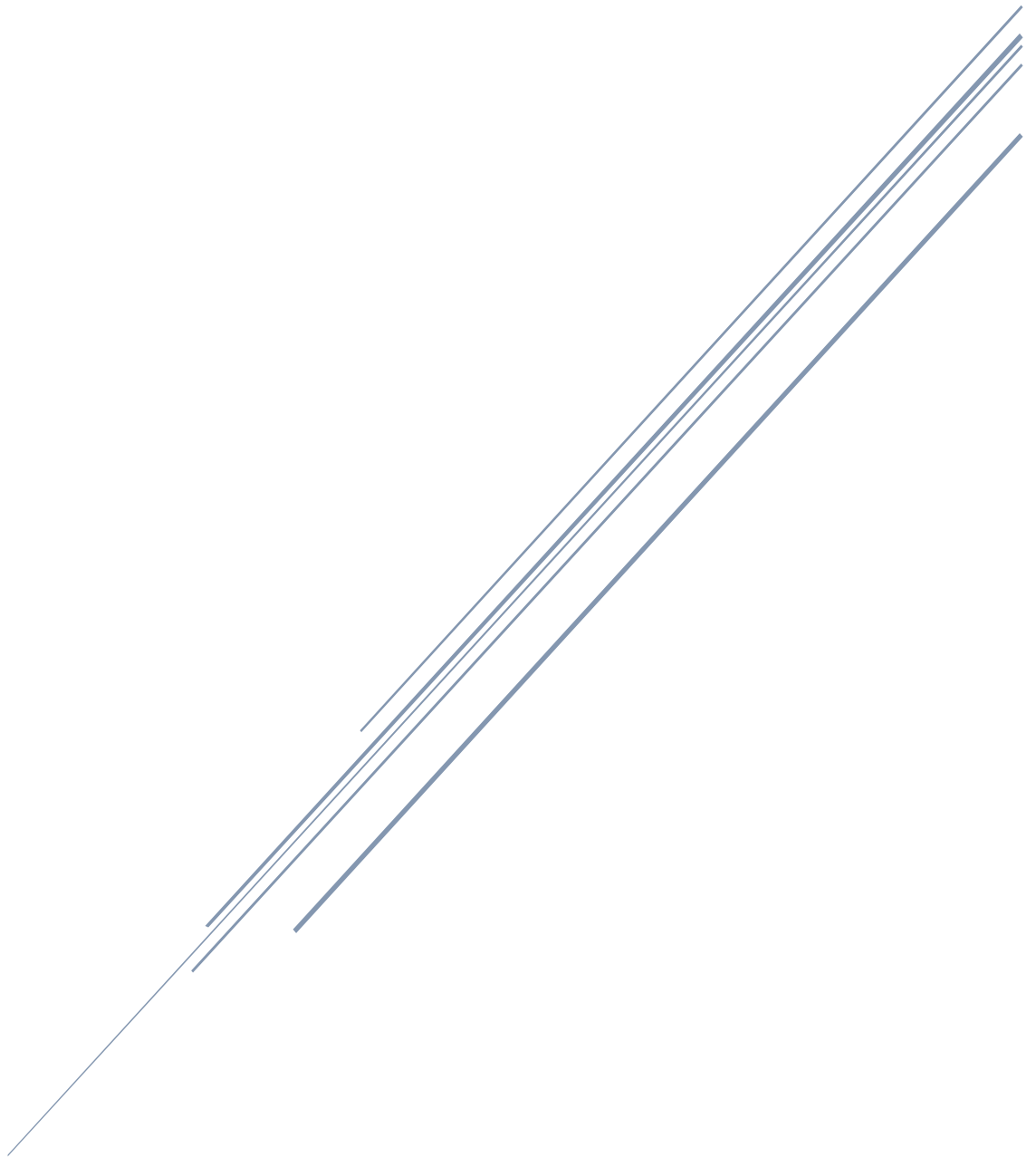


Agentic Financial Advisor Bot for Investment Strategies

Preliminary report



University of London
CM3070 Final Project

Abstract

The financial market presents a challenging environment for investors due to its volatility, high-dimensional data, and overwhelming information flow. This report introduces the design and development of an Agentic Financial Advisor Bot aimed at democratizing access to personalized, transparent investment strategies. The system integrates real-time financial data ingestion, lightweight forecasting via reservoir computing, and decision-making through deep reinforcement learning (FinRL). It further incorporates a natural language interface powered by Qwen3, enabling users to receive investment advice in conversational form. Built within the lightweight agentic framework Smolagents, the bot operates as an explainable, modular, and adaptable portfolio management assistant. Evaluation through backtesting and user feedback will assess the bot's effectiveness in improving investment outcomes and user trust. This project demonstrates how emerging AI technologies can be orchestrated to create accessible, intelligent financial tools for retail investors.

Contents

1.	Introduction.....	1
2.	Project Framework	2
3.	Motivation	3
4.	Literature Review.....	4
4.1.	Forecasting Approaches	4
4.2.	Deep Reinforcement Learning in Trading.....	5
4.2.1.	TD3 for Portfolio Optimization	5
4.2.2.	DDPG for Stock Trading	5
4.3.	Benchmarking and Data Environments.....	6
4.3.1.	FinRL-Meta: Modular Environments and Benchmarks	6
4.3.2.	Dynamic Datasets for Continual FinRL Training.....	6
4.4.	Multimodal Learning for Movement Prediction	7
4.5.	Comparative Analysis and Research Gaps.....	8
4.5.1.	Comparative Overview	8
4.5.2.	Research Gaps and Open Questions	8
4.6.	Conclusion	9
5.	Project Design.....	11
5.1.	Domain and Users	11
5.2.	Design Justification	11
5.3.	Project Structure	12
5.4.	Technologies and Methods	14
5.5.	Work Plan	14
5.6.	Evaluation Plan	16
5.7.	Contingency Plan	18
6.	Feature Prototype	19
6.1.	Forecasting with Reservoir Computing	19
6.2.	Reinforcement Learning with FinRL	21
6.3.	Natural Language Output with Smolagents + Qwen3.....	24
6.4.	Limitations and Planned Enhancements.....	25
7.	Conclusion	27
8.	References	28

1. Introduction

The financial market is a complex, dynamic environment characterized by high volatility, non-stationary signals, and a low signal-to-noise ratio. Accurately predicting asset prices or optimizing investment strategies remains a formidable challenge, attracting the attention of both researchers and practitioners. Traditional models like ARIMA and linear regressions often fall short in capturing nonlinear patterns and adapting to new market conditions.

This report outlines the design and development of an Agentic Financial Advisor Bot, built to support active portfolio management through a combination of financial data processing, predictive modelling, reinforcement learning and natural language interfaces. The system is designed to be adaptable, explainable, and lightweight, providing accessible investment recommendations with a clear rationale and transparency.

2. Project Framework

The project is based on the “Financial Advisor Bot” template. The main objective is to perform active portfolio management. To meet this goal, the system integrates:

- **Data Sources:** Live financial data from Yahoo Finance API, Bloomberg API, and CCXT.
- **Predictive Models:** Short-term asset forecasting is performed using reservoir computing models.
- **Reinforcement Learning Environment:** The FinRL framework [1] is used for strategic decision-making and portfolio allocation.
- **Language Models:** Qwen3 for generating human-readable investment advice.
- **Agentic Framework:** Smolagents [2]—a minimalist, lightweight Python library—is used to compose agentic workflows that interpret user instructions and call relevant functions.
- **Evaluation:** Backtesting for historical comparison and accuracy assessment.

This stack allows the system to perform daily predictions, plan for investment strategy, communicate them clearly, and evaluate performance.

3. Motivation

Financial decision-making remains difficult for many individual investors due to persistent structural challenges. These include:

- **Information overload:** Users are confronted with a constant stream of market data that is difficult to interpret or act upon in real time.
- **High cost of advisory services:** Traditional financial advisors and active portfolio managers are often prohibitively expensive, especially for younger or lower-capital investors.
- **Limitations of existing tools:** Many widely available platforms are passive, offer little to no personalisation, and provide minimal transparency into how investment suggestions are generated.

Modern users increasingly seek investment tools that are intuitive, adaptive to changing market conditions, and capable of offering explainable, trustworthy recommendations. This project addresses these needs by proposing an intelligent, user-oriented agentic system designed to support active portfolio management with modular transparency.

4. Literature Review

In response, the recent surge in machine learning, particularly **deep learning** and **reinforcement learning (RL)**, has opened new avenues for financial forecasting and trading automation. These approaches bring the potential to model intricate temporal dependencies, adapt through online learning, and incorporate high-dimensional, multimodal data sources such as financial news, social media sentiment, and real-time prices.

This review synthesizes six papers that span three emerging fronts in financial AI:

1. **Time series forecasting** using lightweight neural architectures.
2. **Portfolio management and trading** using deep reinforcement learning.
3. **Data-centric and benchmark-driven platforms** for developing and evaluating financial AI agents.

Together, these works represent the convergence of model innovation, infrastructure development, and data integration—setting the stage for robust, adaptive, and scalable financial decision-making systems.

4.1. Forecasting Approaches

Traditional time series forecasting in finance has heavily relied on statistical models such as ARIMA, which assume linearity and stationarity—assumptions frequently violated in real-world financial data. In response, researchers have investigated machine learning-based alternatives that can capture non-linear dependencies and better accommodate the stochastic nature of asset prices.

A notable contribution in this space is the application of **Reservoir Computing (RC)**, particularly **Echo State Networks (ESNs)**, for financial forecasting. Kumar [3] demonstrates that RC can outperform deep learning models such as Long Short-Term Memory (LSTM) networks in both predictive accuracy and computational efficiency. Unlike LSTMs, which require extensive training of recurrent connections, ESNs leverage a randomly initialised dynamic reservoir and train only the output weights—making them significantly more lightweight.

In Kumar's study [3], models were evaluated using daily crude oil prices from 2010 to 2023. The RC model achieved a mean absolute percentage error (MAPE) of 1.57%, significantly outperforming LSTM, which produced a MAPE of 27.69%. Additionally, the ESN completed training in 1.1 seconds, compared to LSTM's 423 seconds. Although the study focuses on crude oil, the underlying price dynamics share characteristics with equity markets, making the results highly relevant for stock forecasting.

While RC models offer advantages in terms of speed and accuracy, they are primarily suited to univariate forecasting tasks. Their adaptability to multivariate or multimodal financial data

remains limited. Nevertheless, they provide a strong, interpretable baseline in scenarios that demand fast, resource-efficient predictions.

4.2. Deep Reinforcement Learning in Trading

While forecasting models aim to predict future prices or asset movements, reinforcement learning (RL) offers a more action-oriented framework—enabling agents to learn strategies that maximise long-term returns. In financial contexts, RL has been widely applied to portfolio optimisation, asset allocation, and automated trading. Two prominent algorithms—**Twin Delayed Deep Deterministic Policy Gradient (TD3)** and **Deep Deterministic Policy Gradient (DDPG)**—are examined in the literature, offering complementary insights into practical RL-based trading.

4.2.1. TD3 for Portfolio Optimization

In their study on portfolio selection, Jiang et al. [4] propose an enhanced TD3 model that incorporates investor-specific constraints such as risk aversion and transaction costs. The agent is trained to optimise a utility function inspired by Markowitz’s mean-variance theory while interacting with a simulated environment based on DJIA and S&P100 constituents.

Empirical results show that the TD3 agent achieves annualised returns of up to 42% and Sharpe ratios exceeding 1.4, outperforming both traditional portfolio strategies (e.g., minimum-variance portfolios) and DRL baselines such as PPO and standard DDPG. The TD3 model is especially effective in high-dimensional asset spaces, where conventional optimisation techniques often falter due to noise and overfitting.

4.2.2. DDPG for Stock Trading

In a complementary study, Liu et al. [5] present a practical DDPG-based trading framework. The agent learns continuously while interacting with an environment composed of 30 DJIA stocks, using historical data from 2009 to 2018. The objective is to maximise cumulative portfolio value while adapting to market dynamics.

The DDPG agent significantly outperforms both the DJIA index and a minimum-variance benchmark, achieving an annualised return of 25.87% and a Sharpe ratio of 1.79. A key advantage of DDPG lies in its ability to operate within continuous action spaces, enabling fine-grained control over asset allocations.

Together, these studies illustrate the growing maturity of deep RL in financial applications. While DDPG excels in online, single-agent trading scenarios, TD3 is particularly well-suited to risk-aware portfolio management. Both approaches highlight the importance of reward shaping, training stability, and realistic simulation environments in the successful deployment of financial RL agents.

4.3. Benchmarking and Data Environments

While algorithmic advances in financial reinforcement learning (FinRL) are promising, they are often evaluated on non-standard datasets or within custom environments, making comparability and reproducibility difficult. Two recent contributions address this gap by introducing benchmarking platforms and dynamic data pipelines that enable realistic and standardised evaluation of FinRL models.

4.3.1. FinRL-Meta: Modular Environments and Benchmarks

In *FinRL-Meta: Market Environments and Benchmarks for Data-Driven Financial Reinforcement Learning*, Liu et al. [6] introduce a comprehensive RLOps platform that automates the end-to-end process of building and evaluating FinRL agents. FinRL-Meta supports a wide range of financial tasks—including stock trading, portfolio management, and multi-agent liquidation—implemented as OpenAI Gym-style environments.

The platform leverages the DataOps paradigm to automate data ingestion, cleaning, feature engineering, and simulation setup. It supports a variety of financial markets (stocks, ETFs, crypto, forex) and integrates multiple data modalities, including technical, fundamental, sentiment, and alternative features (e.g., ESG, patents, social media).

Critically, FinRL-Meta reproduces benchmarks for popular DRL algorithms such as DDPG, PPO, and A2C, offering reproducible baselines for both academic and industrial applications. Ensemble methods further boost performance; for example, ensemble-PPO+A2C+DDPG achieves a Sharpe ratio of 1.53, outperforming individual agents and traditional index strategies.

4.3.2. Dynamic Datasets for Continual FinRL Training

Extending FinRL-Meta, Liu et al. [7] address the limitations of static datasets in RL training by proposing a dynamic data pipeline that enables continual learning. Static datasets often lead to overfitting and limited generalisation. This framework introduces a rolling-window training-testing-trading pipeline, allowing agents to adapt incrementally as new data becomes available.

The system supports real-time data ingestion from over 30 financial sources and is compatible with major RL libraries such as Stable-Baselines3, RLlib, and ElegantRL. It also integrates with paper trading services (e.g., Alpaca) to support minute-level simulation.

This dynamic pipeline enhances realism, reduces survivorship bias, and enables curriculum learning by gradually increasing task complexity. Experiments show that ensemble agents consistently outperform single-policy models and traditional financial benchmarks in this framework.

Together, these two contributions mark a shift from isolated model development to full-stack infrastructure in FinRL—enabling robust pipelines, standardised benchmarks, and scalable simulation environments. Such system-level thinking is essential for bridging the gap between simulation and real-world deployment in financial AI.

4.4. Multimodal Learning for Movement Prediction

In financial markets, asset movements are often influenced by unstructured external signals such as breaking news, investor sentiment, and public discourse on social media. While price-based models capture historical patterns, they frequently miss real-time reactions to such external events. To address this limitation, researchers have turned to **multimodal learning**, integrating textual and numerical data for improved predictive performance.

A pioneering contribution in this domain is the work by Xu and Cohen [8], who introduce **StockNet**, a deep variational generative model that fuses tweets and historical price data to predict binary stock movements (up/down).

Unlike traditional discriminative models, StockNet learns latent market dynamics using recurrent, continuous latent variables trained via variational inference. The model comprises three main components:

1. A **Market Information Encoder** that processes tweet embeddings and historical price changes,
2. A **Variational Movement Decoder** that models stochastic dependencies,
3. An **Attentive Temporal Auxiliary** module that incorporates prior movement predictions to enhance temporal coherence.

The model is trained on a dataset covering 88 U.S. stocks from 2014 to 2016, using Twitter messages and Yahoo Finance price data. It achieves a prediction accuracy of 58.2%, outperforming several baselines—including ARIMA, random forests, and neural NLP models such as the Hierarchical Attention Network (HAN). Notably, StockNet also achieves a higher **Matthews Correlation Coefficient (MCC)**, indicating greater robustness in imbalanced prediction tasks.

StockNet's performance highlights the value of combining social sentiment with technical indicators. It also demonstrates that deep generative models—traditionally applied in language modelling—can effectively capture the uncertainty and volatility inherent in financial markets.

However, the model focuses solely on binary movement prediction and does not address portfolio-level optimisation or capital allocation. Nonetheless, it lays a foundation for future FinRL agents that incorporate multimodal market signals into their policy learning.

4.5. Comparative Analysis and Research Gaps

The six reviewed papers cover a broad and complementary range of financial AI research—from forecasting and trading strategies to infrastructure and data integration. Collectively, they illustrate how modern techniques in **deep learning**, **reinforcement learning**, and **generative modelling** are transforming the way we approach financial prediction and decision-making.

4.5.1. Comparative Overview

Category	Paper(s)	Key Strengths	Limitations
Forecasting	Reservoir Computing [3]	High accuracy, low latency, simple implementation	Single-asset only; no adaptive learning
Trading Agents	DDPG [5], TD3 [4]	Optimised capital allocation, risk sensitivity, outperform traditional methods	Require careful tuning; sensitive to data realism
Infrastructure	FinRL-Meta [6], Dynamic Datasets [7]	Reproducibility, modularity, live data support, benchmark environments	Require computational infrastructure; steep learning curve
Multimodal Modelling	StockNet [8]	Incorporates sentiment & price, generative model with temporal structure	Binary-only prediction, not integrated with RL

These approaches vary in their scope (single asset vs. multi-asset portfolio), output type (prediction vs. policy), and data modalities (price history, sentiment, fundamental signals). Yet they all converge on the broader need for financial AI systems that are more robust, adaptive, and explainable—capable of handling real-world complexity with transparency and reliability.

4.5.2. Research Gaps and Open Questions

Based on the comparative synthesis, several key research gaps emerge:

- **Hybrid Modelling:** Most models emphasise either forecasting (e.g., RC [3], StockNet [8]) or decision-making (e.g., TD3 [4], DDPG [5]). Bridging the two—such as integrating StockNet-like sentiment encoders into RL agents—remains largely unexplored.

- **Explainability:** Despite promising performance, the reviewed models generally lack interpretability. In finance, black-box systems face resistance due to regulatory, ethical, and fiduciary concerns. Future work should prioritise explainable reinforcement learning and feature attribution methods.
- **Multi-Agent Systems:** Financial markets are inherently multi-agent environments. However, most reviewed studies adopt single-agent frameworks. Extending these models to competitive or cooperative **multi-agent RL** settings could offer more realistic and robust simulations of market behaviour.
- **Cross-Modal Reasoning:** StockNet [8] highlights the value of integrating social sentiment with price data. Expanding this approach to include additional modalities—such as financial news, macroeconomic indicators, ESG scores, or patent activity—could enhance robustness and signal diversity.
- **Real-Time Deployment and RLOps:** Although recent works address data pipelines and benchmarks [6], [7], few models are deployed in live markets or evaluated in production environments. Bridging the gap between research prototypes and real-time, scalable systems remains a major challenge in financial AI.

These research gaps highlight promising directions for advancing financial AI systems that are not only accurate, but also robust, transparent, and suitable for real-world deployment.

4.6. Conclusion

This literature review has examined six influential contributions at the intersection of deep learning, reinforcement learning, and financial prediction. These works illustrate major progress across three key fronts:

1. **Forecasting innovations:** Reservoir Computing [3] offers efficient alternatives to deep models such as LSTM, enabling fast and accurate univariate asset prediction.
2. **Reinforcement learning for decision-making:** Methods like TD3 [4] and DDPG [5] demonstrate the capacity of AI agents to make risk-aware portfolio decisions and outperform traditional baselines.
3. **Data-centric and infrastructure advancements:** Platforms such as FinRL-Meta [6] and dynamic data pipelines [7] are advancing reproducibility, standardisation, and readiness for real-time deployment.

In addition, the integration of **multimodal data sources**, as pioneered by StockNet [8], demonstrates the growing importance of combining sentiment and numerical indicators to better capture market behaviour.

Despite these advancements, important challenges remain—particularly in model explainability, continual adaptation, and the development of hybrid systems that can both

predict and act. Future research should aim to bridge these gaps through cross-disciplinary approaches that combine robust machine learning techniques with domain expertise in finance.

As the field continues to evolve, we move closer to building adaptive, transparent, and trustworthy AI systems capable of navigating the complexity and volatility of modern financial markets.

5. Project Design

The **Agentic Financial Advisor Bot** is an AI-driven system designed to deliver intelligent, explainable, and adaptive financial advice to investors. It integrates real-time market data processing, predictive modelling, reinforcement learning, and natural language generation to produce clear and actionable investment strategies.

The system supports natural, conversational interaction, enabling users to engage with the bot using plain English. In return, the bot provides customised investment guidance while maintaining transparency and user control. By combining modular AI components with a language-driven interface, the design aims to bridge the gap between algorithmic decision-making and user-centred financial planning.

5.1. Domain and Users

Domain: Financial technology and personal investment advisory.

Users:

- Retail investors seeking low-cost, accessible investment advice.
- Beginner investors overwhelmed by data and lacking financial expertise.
- Users looking for a transparent alternative to black-box robo-advisors.

This project addresses the intersection of personal finance and artificial intelligence by providing an intelligent interface for users to manage their investments more actively and confidently. For example, a 29-year-old retail investor using platforms like Robinhood may benefit from personalised, transparent, and cost-effective advice without the fees associated with traditional financial advisors.

5.2. Design Justification

The project is designed to address three main user challenges:

- **Information Overload:** Users are bombarded with excessive and often conflicting financial information.
- **Advisor Costs:** Traditional financial advisors remain unaffordable for many retail investors.
- **Lack of Adaptability in Tools:** Most existing tools are passive and lack explainability.

The system design—comprising real-time data integration, time-series forecasting, reinforcement learning for strategy selection, and natural language generation—is directly aligned with these user needs. The use of an **agentic framework** ensures adaptability and interpretability, while integration with public APIs and large language models (LLMs) enhances accessibility and transparency. Each component was selected to optimise for modularity, responsiveness, and explainability.

5.3. Project Structure

The system is designed as a modular, agentic pipeline consisting of six core components. Each module handles a specific function in the forecasting, decision-making, and communication process:

1. Perception:

- Retrieves and preprocesses live or historical financial data using APIs such as Yahoo Finance. This includes normalization, resampling, and feature engineering to prepare the input for downstream models.

2. Reasoning:

- Applies predictive models—such as Reservoir Computing (RC)—to forecast short-term price movements or trends based on recent market behaviour.

3. Action Selection:

- Uses reinforcement learning, implemented via the FinRL framework, to determine optimal portfolio allocations based on predicted market conditions and observed features.

4. Natural Language Reporting:

- Converts technical outputs into user-facing, conversational explanations using the Qwen3 language model. This module provides transparency and improves accessibility for non-expert users.

5. Self-Evaluation:

- Logs predictions and actions for each episode and evaluates performance using backtesting tools such as pyfolio. This helps refine policy model.

6. Agentic Framework:

- Smolagents is used to coordinate the components via natural language-based function calling. It enables flexible orchestration of forecasting, strategy, and reporting modules in response to user queries.

Control Flow and Interaction Design

The bot operates in an agentic fashion:

- The **user interacts with the LLM interface** (Qwen3), asking questions or requesting investment recommendations in natural language.
- **Smolagents parses the user intent** and determines whether additional tools are needed (e.g., fetching market data, generating forecasts, calling the RL policy).

- The required modules are then triggered in sequence. For example, a user asking “What should I do with AAPL today?” results in:
 - pulling recent AAPL data (Perception),
 - generating a prediction (Reasoning),
 - running the RL policy (Action),
 - and returning a human-readable recommendation (Natural Language Reporting).

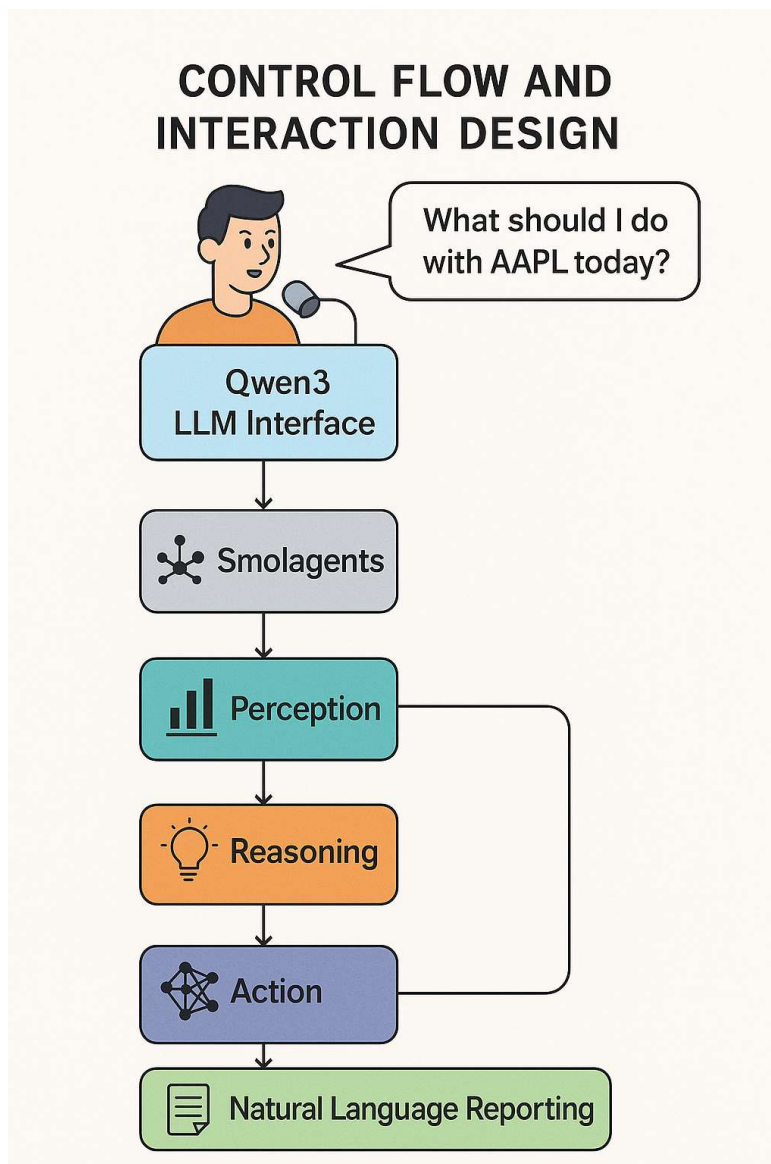


Figure 1. Control Flow

This modular design supports interpretability, flexibility, and extensibility—enabling future upgrades such as replacing the RC module with a Transformer-based model, or substituting Qwen3 with another large language model (e.g., ChatGPT), without altering the core architecture.

In future iterations, the agentic framework may be extended with **user memory capabilities**, allowing the system to retain contextual information such as portfolio history, risk tolerance, and user-defined investment goals across sessions. This would enable more personalised, context-aware recommendations aligned with the user’s long-term financial behaviour.

To support such functionality, **stigmergic reinforcement learning** [10] is being explored as a modular upgrade. As a decentralised RL paradigm, stigmergic learning enables independent agents to specialise and coordinate through a shared environment or memory layer. This architecture is well-suited to incorporating personalised strategies and dynamically adapting to user-specific objectives.

Such an approach would support increased customisation and responsiveness, particularly in scenarios where user portfolios fall outside the DOW30 index. Currently, the system is limited to DOW30 stocks and cannot dynamically adapt to index changes—such as the replacement of INTC with NVDA. A modular RL design, as offered by stigmergic learning, could enhance system robustness and scalability across diverse asset universes.

Additionally, future versions will incorporate an **automated retraining pipeline** that periodically updates forecasting and strategy models using the most recent market data. This is essential in non-stationary financial environments, where model performance can degrade rapidly. Scheduled retraining ensures continued model relevance and responsiveness to regime shifts or rare market events, without the need for manual retraining.

5.4. Technologies and Methods

- **Reservoir Computing:** Time-series prediction model with low computational cost.
- **FinRL [1]:** Reinforcement learning framework for financial trading with built in Reinforcement Learning and backtesting feature.
- **Qwen3:** A family of open-source large language models, with the lightweight variant used in this project.
- **Smolagents [2]:** Lightweight agentic system framework.
- **Yahoo Finance, Bloomberg, CCXT APIs:** Live financial data.

5.5. Work Plan

To accelerate early development, simpler models such as **Advantage Actor-Critic (A2C)** for reinforcement learning and standard **Reservoir Computing (RC)** for forecasting are employed in the initial prototype. However, these are not intended for the final version. The system has been architected to support modular upgrades; future iterations will incorporate more advanced components, such as **DDPG** or **PPO** for trading, and **Next-Generation Reservoir Computing (NG-RC)** for forecasting [9].

As part of the reinforcement learning upgrade, the final system may also integrate concepts from recent work on **stigmergic multi-agent learning**, particularly mechanisms for decentralised coordination and federated training. This direction is inspired by Xu et al. [10] and is expected to improve robustness in portfolio strategies under dynamic market conditions.

The following Gantt chart outlines the planned phases of development. The primary objective is to deliver a working system pipeline by Week 12. Individual component testing is scheduled for Week 11. The coordinated system will use **Smolagents** to integrate RC-based forecasts, FinRL-generated portfolio strategies, and Qwen3-based explanations into a cohesive agentic feedback loop.

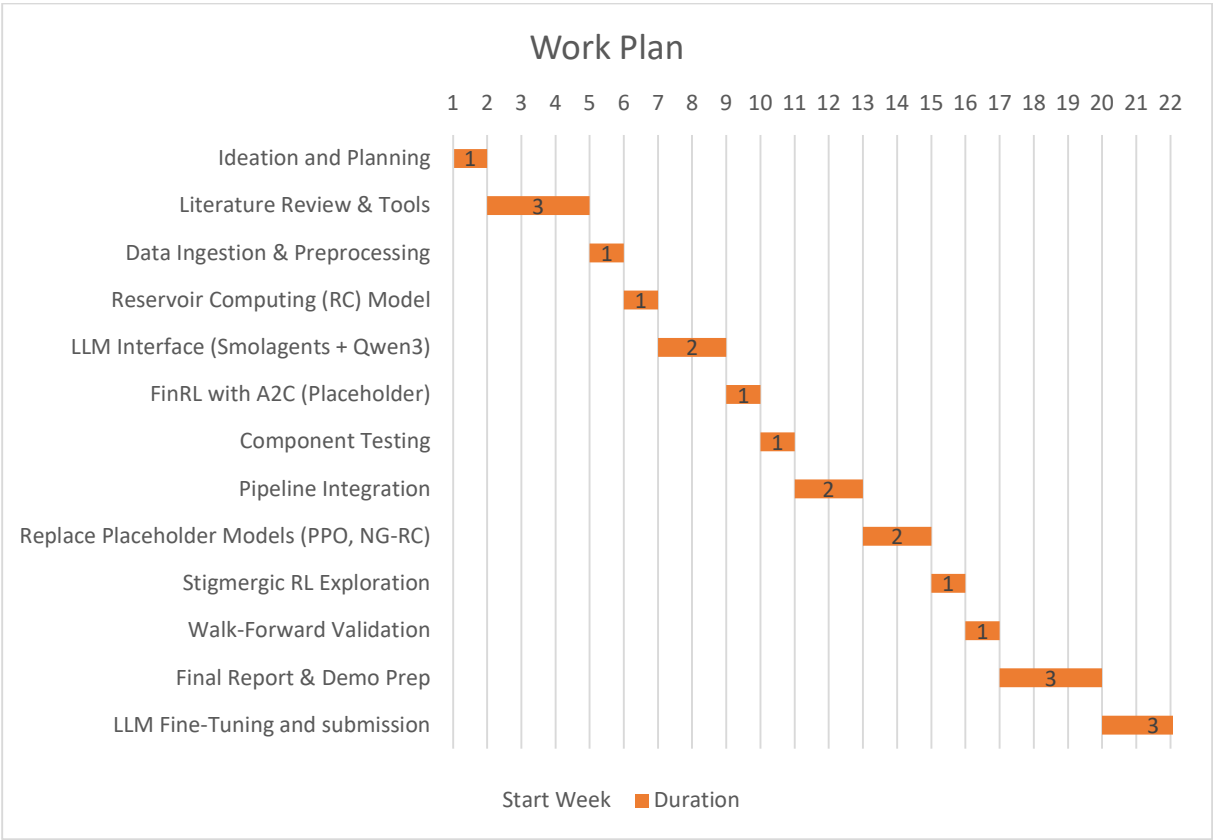


Figure 2. Work Plan Gantt Chart

Week	Task
1	Ideation and Planning
2-4	Literature review, finalize tools and technologies
5	Prototype data ingestion and preprocessing module
6	Develop and Test RC Models (based on Kumar, 2023)
7-8	Connect LLM (Qwen3) with Agentic Framework (Smolagents) for NL Output

9	Test FinRL for Action Strategy Generation and Backtesting
10	Internal Testing of Individual Components and preliminary report submission
11-12	Integrate Components (RC + FinRL + LLM) into a Unified Pipeline and explore possible user-friendly web interface
13	Replace Placeholder Models (e.g., A2C → PPO, RC → NG-RC)
14-15	Explore Stigmergic RL Architecture (based on Xu et al., 2020)
16	Evaluate System Performance (Quantitative & Qualitative Metrics), Stress Testing with Market Volatility Scenarios
17-19	Prepare Final Report, Slides, and Video Presentation
20-22	LLM fine-tuning, polishing and final submission

5.6. Evaluation Plan

The evaluation of the prototype system focuses on three core modules: forecasting, reinforcement learning-based decision-making, and natural language interaction. Each is tested independently due to the modular design, with metrics tailored to the functionality of each component.

Forecasting Module (Reservoir Computing)

The RC model is evaluated on its ability to predict the next-day closing price of AAPL using historical data. The evaluation is based on a 100-step forecast horizon, using the following metrics:

- **Mean Absolute Error (MAE)**
- **Root Mean Squared Error (RMSE)**
- **Mean Absolute Percentage Error (MAPE)**
- **R-squared (R^2)**
- **Normalized RMSE (based on mean and range)**

A baseline performance is compared against a naive predictor (e.g., yesterday's price = today's prediction). The model is considered effective if it significantly outperforms the baseline in stable market conditions and degrades predictably in volatile regimes.

Reinforcement Learning Module (FinRL + A2C)

The A2C agent is evaluated using **backtesting** on unseen data (July 2020 to October 2021) in a simulated trading environment. The following financial and performance metrics may be used:

- **Cumulative return**
- **Sharpe ratio**
- **Maximum drawdown**
- **Volatility of returns**
- **Portfolio allocation stability**

Comparisons are made against a passive buy-and-hold strategy of the DOW30 index. Due to A2C's placeholder status, the focus is on functional correctness and responsiveness to market signals rather than peak performance.

Natural Language Module (Smolagents + Qwen3)

This component is evaluated qualitatively by:

- Demonstrating task routing accuracy via Smolagents
- Verifying that queries such as *"What should I do with AAPL today?"* return coherent and relevant responses
- Checking consistency and tone of Qwen3 output under various inputs

In future iterations, this module may be evaluated through **user studies** or structured feedback from simulated dialogue sessions to assess response helpfulness, trustworthiness, and transparency.

Evaluation Limitations

- Integration between modules is not yet complete; evaluations are conducted in isolation.
- Backtesting does not currently simulate transaction costs, slippage, or market impact.
- No walk-forward validation or cross-validation has been applied yet.

Future Improvements

The evaluation plan will be extended to include:

- Rolling retraining for forecasting models
- Real-time testing on live or delayed market data
- Task success scoring for the LLM based on user satisfaction

- Comparative testing across multiple RL models (e.g., PPO, TD3) under identical simulated conditions

5.7. Contingency Plan

To ensure continued functionality in the face of technical challenges, several contingency measures have been defined:

- If integration with the large language model (LLM) fails, the system will fall back to using **templated natural language responses**. These will be generated programmatically based on structured model outputs, ensuring that user interaction remains interpretable even without LLM support.
- If training or tuning of the reinforcement learning (RL) agent proves unsuccessful—due to convergence issues or suboptimal performance—the system will revert to a **rule-based or pre-trained policy**. This policy may rely on simple heuristics such as historical moving averages or momentum indicators to produce basic asset allocation recommendations.
- As real-time backtesting requires significant historical context and consistent live data availability, the evaluation and demonstration of the prototype will rely on **pre-collected historical datasets**. These datasets ensure that testing can be conducted consistently and without dependency on market hours or connectivity

These fallback strategies allow each component to function independently and maintain demonstrable output, even if specific subsystems underperform or fail.

6. Feature Prototype

This section presents a modular demonstration of the key components developed for the Agentic Financial Advisor Bot. While full system integration is still underway, each module has been implemented and tested individually to validate its functionality and design.

The prototype currently consists of three core components:

1. **Forecasting Module:** A Reservoir Computing (RC) model trained on historical stock data to predict next-day prices. Its simplicity and speed make it well-suited for frequent and rapid retraining in volatile markets.
2. **Strategy Module:** A reinforcement learning agent implemented using the FinRL framework. Currently, a lightweight **Advantage Actor-Critic (A2C)** algorithm is used as a placeholder to test environment setup and action selection. **It is not intended for the final submission**, and will be replaced with a more advanced model in later iterations.
3. **Language Interface Module:** A natural language interface built using Smolagents and Qwen3, enabling users to interact with the system through conversational queries and receive readable, explainable advice.

Each component is designed to be lightweight, interpretable, and modular—allowing future upgrades and replacements without disrupting the entire pipeline. This section walks through the current implementation of each part, along with selected outputs, design motivations, and key limitations.

6.1. Forecasting with Reservoir Computing

We begin with a lightweight time-series forecasting module using **Echo State Networks (ESNs)**, a form of Reservoir Computing (RC). This approach was chosen for its:

- **Fast training speed:** only the output weights are trained via ridge regression,
- **Good performance on univariate financial data**, and
- **Simplicity in setup**, making it an effective baseline for real-time applications [3].

In this prototype, we focus on forecasting the next-day **closing price of AAPL (Apple Inc.)**, using historical price data spanning **from January 2, 2008 to June 12, 2025**. The model is trained on the **first 1000 normalised closing price values**, using a **window of 10 previous time steps as input**.

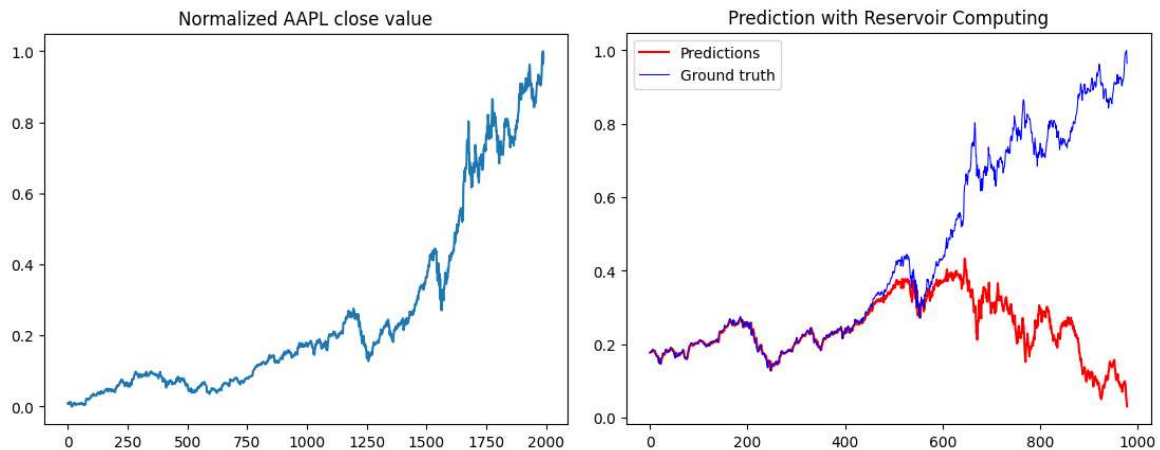


Figure 3.
Left: Normalised AAPL close value used to train and test RC model
Right: Actual vs. predicted AAPL prices using RC model.

The dataset was normalized to [0, 1] and partitioned into:

- **Training set:** 1000 time steps (2008–2011),
- **Evaluation:** following 900+ steps, extending into periods of significant volatility.

As shown in **Figure 3**, the RC model tracks the actual closing prices closely for the first ~400 forecast steps. However, its accuracy deteriorates significantly during a period of rapid and uncharacteristic growth in AAPL stock value. This pattern illustrates a common limitation of static forecasting models—they tend to underperform when the underlying data distribution shifts substantially, such as during earnings shocks, macroeconomic disruptions, or black swan events like the COVID-19 pandemic.

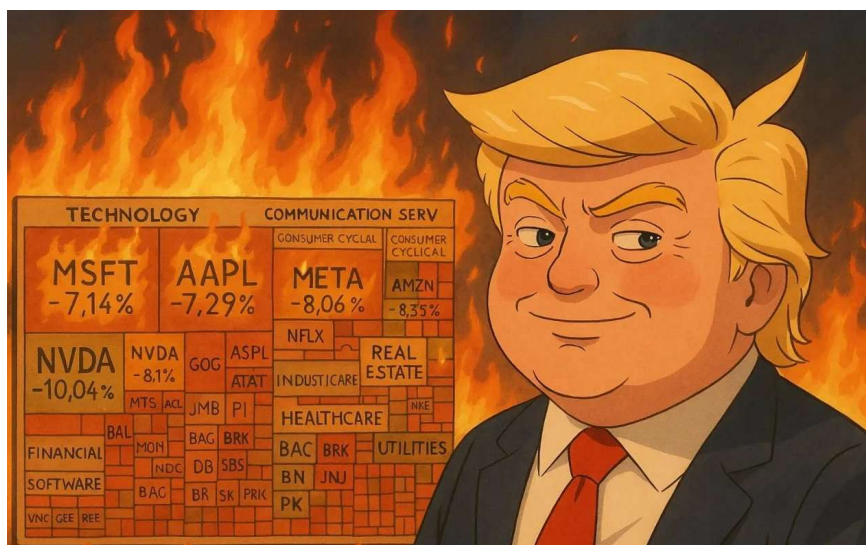


Figure 4. Meme illustrating a sudden market downturn following tariff announcements.
Source: X.com (formerly Twitter), April 2025.

Previous work has demonstrated similar outcomes in financial forecasting using RC models. For instance, Kumar [3] reports that RC outperforms LSTM in both training efficiency and predictive accuracy for univariate price data, but also notes performance degradation under shifting market regimes.

To quantify performance, we evaluate the model using several standard error metrics:

- **Mean Absolute Error (MAE):** 0.2056
- **Root Mean Squared Error (RMSE):** 0.3467
- **Mean Absolute Percentage Error (MAPE):** 27.33%
- **R-squared (R^2):** -0.7052
- **Mean Squared Error (MSE):** 0.1202
- **Normalized RMSE (based on mean):** 0.7776
- **Normalized RMSE (based on max - min):** 0.3975

The **negative R^2 value** indicates that, within this test period, the RC model performs worse than a naïve mean predictor—particularly during non-stationary phases. These results reinforce the need for frequent retraining or the adoption of more robust architectures, such as **Next-Generation Reservoir Computing (NG-RC)** [9], which offer improved adaptability to regime shifts and long-term dependencies.

6.2. Reinforcement Learning with FinRL

To automate portfolio allocation, we model the stock trading process as a **Markov Decision Process (MDP)** and optimise it using **Deep Reinforcement Learning (DRL)**. In this framework, the agent interacts with a simulated financial environment and learns to allocate capital across a portfolio to maximize cumulative returns over time. This implementation is based on the FinRL framework described by Liu et al. [1].

MDP Formulation

- **Action Space** ($a \in (-1, 1)$): The agent outputs a portfolio weight vector $[a_1, a_2, \dots, a_n]$, where a_i represents the proportion of capital allocated to the i^{th} asset. In this prototype, weights are constrained to $a_i \in [0, 1]$ (i.e., long-only), and $\sum a_i = 1$.
- **State Space:** The agent observes a set of financial indicators per asset, including Open, High, Low, Close prices, volume, and a suite of technical indicators (e.g., MACD, RSI, ADX). This mirrors the kind of market data a human investor would consider before making a trade.
- **Reward Function:** The reward is defined as the change in total portfolio value:

$$r(s, a, s') = v' - v,$$

where v and v' represent the portfolio values at states s and s' , respectively. This directly incentivizes the agent to increase portfolio value through its actions.

- **Environment:** The environment includes the **30 constituent stocks of the Dow Jones Industrial Average (DOW30)**. Historical stock data is sourced from the Yahoo Finance API.
-

Dataset and Preprocessing

- **Training period:** January 1, 2009 – July 1, 2020
- **Trading period (backtesting):** July 1, 2020 – October 31, 2021
- **Assets:** All 30 stocks in the DOW30 index
- **Data source:** Yahoo Finance (via FinRL data pipeline)
- **Preprocessing:** Performed using FinRL's default pipeline, including:
 - Technical indicators: MACD, RSI, CCI, and ADX
 - Feature scaling and missing value handling

Preprocessing is handled entirely through FinRL's built-in pipeline to ensure reproducibility and consistency. No manual tuning was performed due to limited domain expertise in financial feature selection.

Model Implementation

- **Framework:** FinRL
- **Algorithm:** Advantage Actor Critic (A2C)
- **Episodes:** 50
- **Training Frequency:** Daily
- **Action granularity:** Continuous portfolio weights across 30 assets

The A2C algorithm is used as a **placeholder model** to validate the RL pipeline. It is a policy gradient method that separately learns a value function (critic) and an action policy (actor). Although computationally efficient, A2C may underperform on complex allocation tasks without further tuning.

Portfolio Allocation (Action Output)

During each time step, the agent outputs a **vector of portfolio weights** representing how much capital to allocate to each of the assets in DOW30. These weights sum to 1 and are constrained to be non-negative.

A sample allocation generated by the A2C agent is shown below:

	AAPL	AMGN	AXP	BA	CAT	CRM	CSCO	CVX	DIS	GS	...	MMM
date												
2020-07-01	0.035714	0.035714	0.035714	0.035714	0.035714	0.035714	0.035714	0.035714	0.035714	0.035714	...	0.035714
2020-07-02	0.039087	0.021217	0.036143	0.021217	0.041956	0.044596	0.021217	0.056761	0.021217	0.043837	...	0.029670
2020-07-06	0.039087	0.021217	0.036143	0.021217	0.041956	0.044596	0.021217	0.056761	0.021217	0.043837	...	0.029670
2020-07-07	0.039093	0.021209	0.036193	0.021209	0.041960	0.044606	0.021209	0.056802	0.021209	0.043838	...	0.029670
2020-07-08	0.039087	0.021217	0.036143	0.021217	0.041956	0.044596	0.021217	0.056761	0.021217	0.043837	...	0.029670

Figure 5. Sample portfolio allocation by A2C agent.

This vector is the **core action** taken by the agent, and it is used to simulate market interaction in the environment. At the next time step, the agent observes the market response (e.g., price changes) and receives a reward based on the updated portfolio value.

Backtesting

Backtesting plays a critical role in evaluating the performance and robustness of an automated trading strategy. It allows us to assess how well the agent would have performed in historical market conditions—before risking real capital.

To reduce human error and ensure consistency, we use an **automated backtesting workflow**, powered by the **Quantopian pyfolio package [11]**. pyfolio provides a rich set of diagnostic tools and visualizations, including:

- Cumulative returns
- Daily returns
- Rolling Sharpe ratio
- Drawdown periods
- Exposure analysis
- Tear sheets for overall performance evaluation

The graph below is a comparison between our model and baseline model from pyfolio

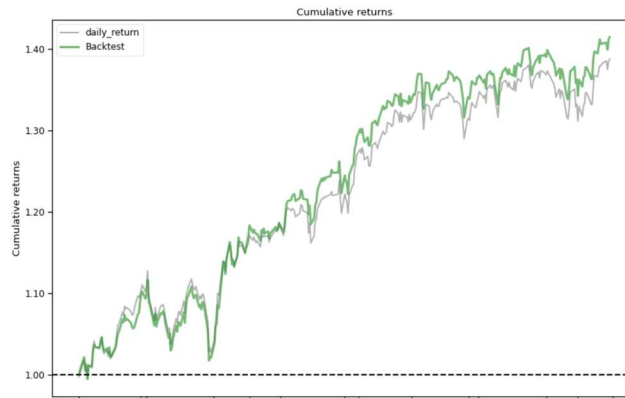


Figure 6. Backtesting performance compared to baseline from pyfolio.

6.3. Natural Language Output with Smolagents + Qwen3

To complete the feedback loop between the system and the user, we integrate a natural language interface powered by **Smolagents** and **Qwen3**. This allows users to interact with the system using natural language queries and receive clear, human-readable explanations of forecasts, strategy decisions, and portfolio recommendations.

Architecture and Design

The language module consists of two main parts:

- **Smolagents Framework:** Handles function routing. It parses the user’s natural language query, identifies the target function (e.g., “get latest data”, “make prediction”, “tell me what to invest in”), and routes it to the appropriate functions.
- **Qwen3 LLM:** Converts structured outputs (e.g., actions, recommendations) into fluent natural language responses. It provides a conversational layer between the agent's logic and the user.

This setup allows the model to behave like a virtual financial assistant, generating explanations based on model output and user queries, making the interaction less intimidating.

A simple example of the system's input-output flow is shown below:

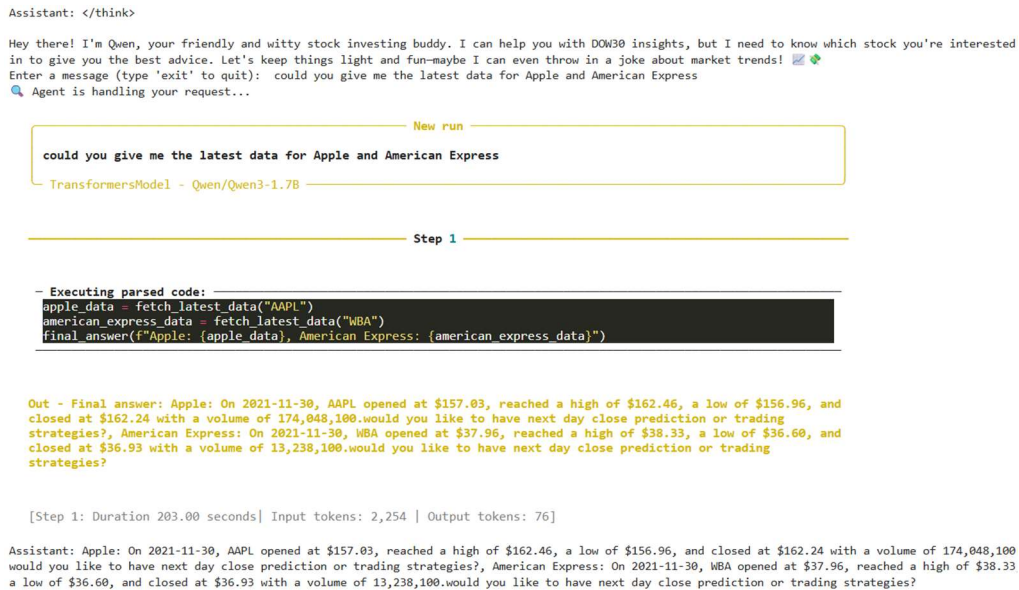


Figure 7. Sample natural language interaction with the system.

This interaction demonstrates how the system converts technical outputs into accessible, user-friendly advice. However, this is far from perfect as the model might hallucinate or confuse user's request. In this example, the model confuses WBA (Walgreens Boots Alliance) and AXP (American Express).

6.4. Limitations and Planned Enhancements

While the prototype demonstrates the individual components of the Agentic Financial Advisor Bot, several limitations remain that will be addressed in future development stages:

Model Limitations

- The **Reservoir Computing (RC)** model, while lightweight and fast, degrades rapidly under distribution shifts. It lacks multivariate inputs and cannot adapt to regime changes without frequent retraining.
- The **A2C reinforcement learning agent** serves as a placeholder and lacks robustness for complex allocation tasks. It currently does not incorporate transaction costs, risk constraints, or exposure limits.
- The **Qwen3 language model** has not been fine-tuned on financial dialogue, and its outputs may be generic or imprecise in some contexts.

Evaluation and Testing Gaps

- No **walk-forward validation** or rolling-window backtesting has been implemented.
- Performance metrics like Sharpe ratio and drawdown are not yet integrated into the reward function.
- The system has only been tested on historical data, with no real-time inference or user trials.

Integration Status

- Components are currently tested in isolation. The end-to-end pipeline is still under development.
- The natural language interface cannot yet trigger real-time data queries or model calls autonomously.

Planned Enhancements

To address the limitations above, the following improvements are planned:

- **Upgrade Strategy Module:** Replace A2C with more stable algorithms such as **TD3** or **PPO** for better control in continuous action spaces.
- **Stigmergic RL Exploration:** Incorporate **stigmergic independent reinforcement learning** for multi-agent portfolio collaboration and distributed decision-making.
- **Improve Forecasting:** Implement **Next-Generation Reservoir Computing (NG-RC)** for more interpretable and adaptive forecasting.
- **Risk-Aware Reward Functions:** Redesign the reward to include **Sharpe ratio**, **drawdown penalties**, and **transaction cost models**.
- **LLM Fine-Tuning:** Fine-tune Qwen3 on a dataset of financial queries and reports for more relevant and trustworthy responses.
- **Pipeline Integration:** Use Smolagents to orchestrate RC, FinRL, and Qwen3 in a single callable chain.
- **Deployment & UI:** Create a web-based interface or CLI assistant for user interaction and daily portfolio suggestions.
- **Automated retraining:** Implement a scheduled retraining pipeline to update forecasting and RL models with recent data, maintaining adaptability under shifting market conditions.

Long-Term Directions

Future iterations may incorporate **user memory** and **portfolio tracking** to support personalised investment recommendations. A modular framework, such as **stigmergic RL**, offers the flexibility needed for these features—especially as the system expands beyond the static DOW30 index to support custom portfolios and dynamic asset universes.

7. Conclusion

This report has presented the modular architecture, technical design, and prototype implementation of an **Agentic Financial Advisor Bot**. The system integrates **Reservoir Computing** for time-series forecasting, **reinforcement learning** for investment strategy generation, and **large language models** for explainable interaction. Together, these components form a lightweight, interpretable decision-making pipeline tailored to retail investors.

Future development will focus on enhancing system robustness under volatile market conditions, fine-tuning the language model for financial dialogue, and exploring **stigmergic multi-agent reinforcement learning** to improve adaptability and decentralised coordination.

Long-term extensions include adding **user-specific memory** and personalised feedback mechanisms, enabling the system to retain preferences, track portfolio performance, and generate more tailored investment advice. Stigmergic RL is expected to play a central role in supporting scalable, modular learning across diverse market scenarios and user-defined asset sets.

The project aims to evolve into a flexible and trustworthy personal financial assistant—capable of dynamic adaptation, transparency, and continual improvement through modular upgrades.

8. References

- [1] Xiao-Yang Liu, Hongyang Yang, Jiechao Gao, and Christina Dan Wang. 2021. FinRL: Deep reinforcement learning framework to automate trading in quantitative finance. In *Proceedings of the 2nd ACM International Conference on AI in Finance (ICAIF '21)*. ACM. <https://doi.org/10.1145/3490354.3494366>
- [2] Aymeric Roucher, Albert Villanova del Moral, Thomas Wolf, Leandro von Werra, and Erik Kaunismäki. 2025. *smolagents: A smol library to build great agentic systems*. <https://github.com/huggingface/smolagents>
- [3] Kaushal Kumar. 2023. *Forecasting crude oil prices using reservoir computing models*. arXiv, preprint arXiv:2306.03052. <https://arxiv.org/abs/2306.03052>
- [4] Yifu Jiang, Jose Olmo, and Majed Atwi. 2024. Deep reinforcement learning for portfolio selection. *Global Finance Journal*, 62, 101016. <https://doi.org/10.1016/j.gfj.2024.101016>
- [5] Xiao-Yang Liu, Zhuoran Xiong, Shan Zhong, Hongyang Yang, and Anwar Walid. 2022. *Practical deep reinforcement learning approach for stock trading*. arXiv, arXiv preprint arXiv:1811.07522. <https://arxiv.org/abs/1811.07522>
- [6] Xiao-Yang Liu, Ziyi Xia, Jingyang Rui, Jiechao Gao, Hongyang Yang, Ming Zhu, Christina Dan Wang, Zhaoran Wang, and Jian Guo. 2022. *FinRL-Meta: Market environments and benchmarks for data-driven financial reinforcement learning*. arXiv,, arXiv preprint arXiv:2211.03107. <https://arxiv.org/abs/2211.03107>
- [7] Xiao-Yang Liu, Ziyi Xia, Hongyang Yang, Jiechao Gao, Daochen Zha, Ming Zhu, Christina Dan Wang, Zhaoran Wang, and Jian Guo. 2023. *Dynamic datasets and market environments for financial reinforcement learning*. arXiv, arXiv preprint arXiv:2304.13174. <https://arxiv.org/abs/2304.13174>
- [8] Yumo Xu and Shay B. Cohen. 2018. Stock movement prediction from tweets and historical prices. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 1970–1979. Association for Computational Linguistics. <https://doi.org/10.18653/v1/P18-1183>
- [9] Daniel J. Gauthier, Erik Bollt, Aaron Griffith, and Wendson A. S. Barbosa. 2021. Next generation reservoir computing. *Nature Communications*, 12, 5564. <https://doi.org/10.1038/s41467-021-25801-2>
- [10] Xing Xu, Rongpeng Li, Zhifeng Zhao, and Honggang Zhang. 2021. *Stigmergic independent reinforcement learning for multi-agent collaboration*. arXiv , arXiv preprint arXiv:1911.12504. <https://arxiv.org/abs/1911.12504>
- [11] Quantopian Inc. 2016. *Pyfolio: A Python library for performance and risk analysis of financial portfolios*. <https://github.com/quantopian/pyfolio>